Relational Query Language in DBMS

- SQL has its own querying methods to interact with the database. But how do these queries work in the database? These queries work similarly to Relational Algebra that we study in mathematics. In the database, we have tables participating in relational Algebra.

  Relational Database systems are expected to be equipped with a query language that assists users to query the database. Relational Query Language is used by the user to communicate with the database user requests for the information from the database. Relational algebra breaks the user requests and instructs the DBMS to execute the requests. It is the language by which the user communicates with the database. They are generally on a higher level than any other programming language. These relational query languages can be Procedural and Non-Procedural.

Types of Relational Query Language

There are two types of relational query language:

- Procedural Query Language
- Non-Procedural Language

Procedural Query Language

In Procedural Language, the user instructs the system to perform a series of operations on the database to produce the desired results. Users tell what data to be retrieved from the database and how to retrieve it. Procedural Query Language performs a set of queries instructing the DBMS to perform various transactions in sequence to meet user requests.

Relational Algebra is a Procedural Query Language

Relational Algebra could be defined as the set of operations on relations.
There are a few operators that are used in relational algebra -

1. **Select (σ sigma):** Returns rows of the input relation that satisfy the provided predicate. It is unary Operator means requires only one operand.
2. **Projection (π):** Show the list of those attribute which we desire to appear and rest other attributes are eliminated from the table. It seperates the table vertically.
3. **Set Difference (-):** It returns the difference between two relations . If we have two relations R and S them R-S will return all the tuples (row) which are in relation R but not in Relation S , It is binary operator.
4. **Cartesian Product (X):** Combines every tuple (row) of one table with every tuple (row) in other table ,also referred as cross Product . It is a binary operator.
5. **Union (U):** Outputs the union of tuples from both the relations. Duplicate tuples are eliminated automatically. It is a binary operator means it require two operands.

1. Selection(σ)

The Selection Operation is basically used to filter out rows from a given table based on certain given condition. It basically allows us to retrieve only those rows that match the condition as per condition passed during SQL Query.
Example: If we have a relation R with attributes A, B, and C, and we want to select tuples where C > 3, we write:

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

$\sigma_{(c>3)}$(R) will select the tuples which have c more than 3.
Output:

A	B	C
1	2	4
4	3	4

Explanation: *The selection operation only filters rows but does not display or change their order. The projection operator is used for displaying specific columns.*

2. Projection(π)

While Selection operation works on rows, similarly projection operation of relational algebra works on columns. It basically allows us to pick specific columns from a given relational table based on the given condition and ignoring all the other remaining columns.

Example: Suppose we want columns B and C from Relation R.
$\pi_{(B,C)}$(R) will show following columns.
Output:

B	C
2	4
2	3

B	C
3	4

Explanation: *By Default, projection operation removes duplicate values.*

3. Union(U)

The Union Operator is basically used to combine the results of two queries into a single result. The only condition is that both queries must return same number of columns with same data types. Union operation in relational algebra is the same as union operation in set theory.

Example: Consider the following table of Students having different optional subjects in their course.

FRENCH

Student_Name	Roll_Number
Ram	01
Mohan	02
Vivek	13
Geeta	17

GERMAN

Student_Name	Roll_Number
Vivek	13
Geeta	17
Shyam	21
Rohan	25

If FRENCH and GERMAN relations represent student names in two subjects, we can combine their student names as follows:

$\pi_{(Student\_Name)}(FRENCH) \cup \pi_{(Student\_Name)}(GERMAN)$
Output:

Student_Name
Ram
Mohan
Vivek
Geeta
Shyam
Rohan

Explanation: *The only constraint in the union of two relations is that both relations must have the same set of Attributes.*

4. Set Difference(-)

Set difference basically provides the rows that are present in one table, but not in another tables. Set Difference in relational algebra is the same set difference operation as in set theory.
Example: To find students enrolled only in FRENCH but not in GERMAN, we write:
$\pi_{(Student\_Name)}(FRENCH) - \pi_{(Student\_Name)}(GERMAN)$

Student_Name
Ram
Mohan

Explanation: *The only constraint in the Set Difference between two relations is that both relations must have the same set of Attributes.*

5. Rename(ρ)

Rename operator basically allows you to give a temporary name to a specific relational table or to its columns. It is very useful when we want to avoid ambiguity, especially in complex Queries. Rename is a unary operation used for renaming attributes of a relation.

Example: We can rename an attribute B in relation R to D

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

$\rho_{(D/B)}$R will rename the attribute 'B' of the relation by "D".

Output Table:

A	D	C
1	2	4
2	2	3
3	2	3
4	3	4

6. Cartesian Product(X)

The Cartesian product combines every row of one table with every row of another table, producing all the possible combination. It's mostly used as a precursor to more complex operation like joins. Let’s say A and B, so the cross product between A X B will result in all the attributes of A followed by each attribute of B. Each record of A will pair with every record of B.

Relation A:

Name	Age	Sex
Ram	14	M
Sona	15	F
Kim	20	M

Relation B:

ID	Course
1	DS
2	DBMS

Output: If relation A has 3 rows and relation B has 2 rows, the Cartesian product A × B will result in 6 rows.

Name	Age	Sex	ID	Course
Ram	14	M	1	DS
Ram	14	M	2	DBMS
Sona	15	F	1	DS
Sona	15	F	2	DBMS
Kim	20	M	1	DS
Kim	20	M	2	DBMS

Explanation: If A has 'n' tuples and B has 'm' tuples then A X B will have 'n*m' tuples.

Non-Procedural Language
In Non Procedural Language user outlines the desired information without giving a specific procedure or without telling the steps by step process for attaining the information. It only gives a single Query on one or more tables to get .The user tells what is to be retrieved from the database but does not tell how to accomplish it.

For Example: get the name and the contact number of the student with a Particular ID will have a single query on STUDENT table. Relational Calculus is a Non Procedural Language .

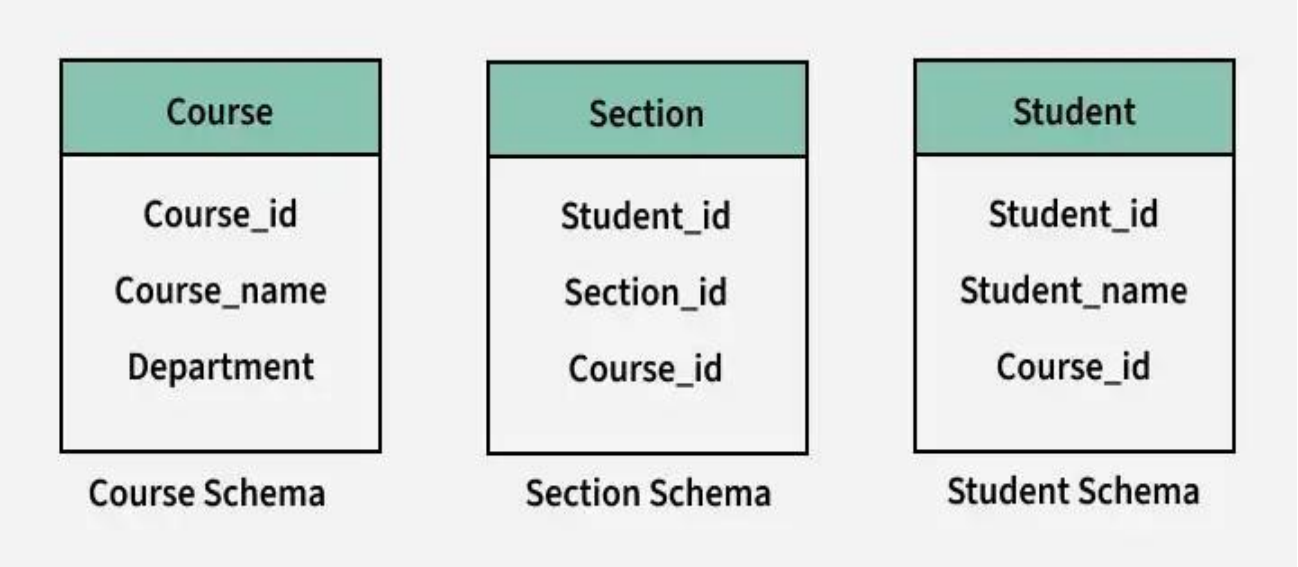
Relational Calculus exists in two forms:

1. **Tuple Relational Calculus (TRC):** Tuple Relational Calculus is a non procedural query language , It is used for selecting the tuples that satisfy the given condition or predicate . The result of the relation can have one or more tuples (row).
2. **Domain Relational Calculus (DRC):** Domain Relational Calculus is a Non Procedural Query Language , the records are filtered based on the domains , DRC uses the list of attributes to be selected from relational based on the condition.

Database Schemas

A database schema defines the structure and organization of data within a database. It outlines how data is logically stored, including the relationships between different tables and other database objects. The schema serves as a blueprint for how data is stored, accessed, and manipulated, ensuring consistency and integrity throughout the system.

What is Schema?
A schema is the blueprint or structure that defines how data is organized and stored in a database. It outlines the tables, fields, relationships, views, indexes, and other elements within the database. The schema defines the logical view of the entire database and specifies the rules that govern the data, including its types, constraints, and relationships.



Schemas

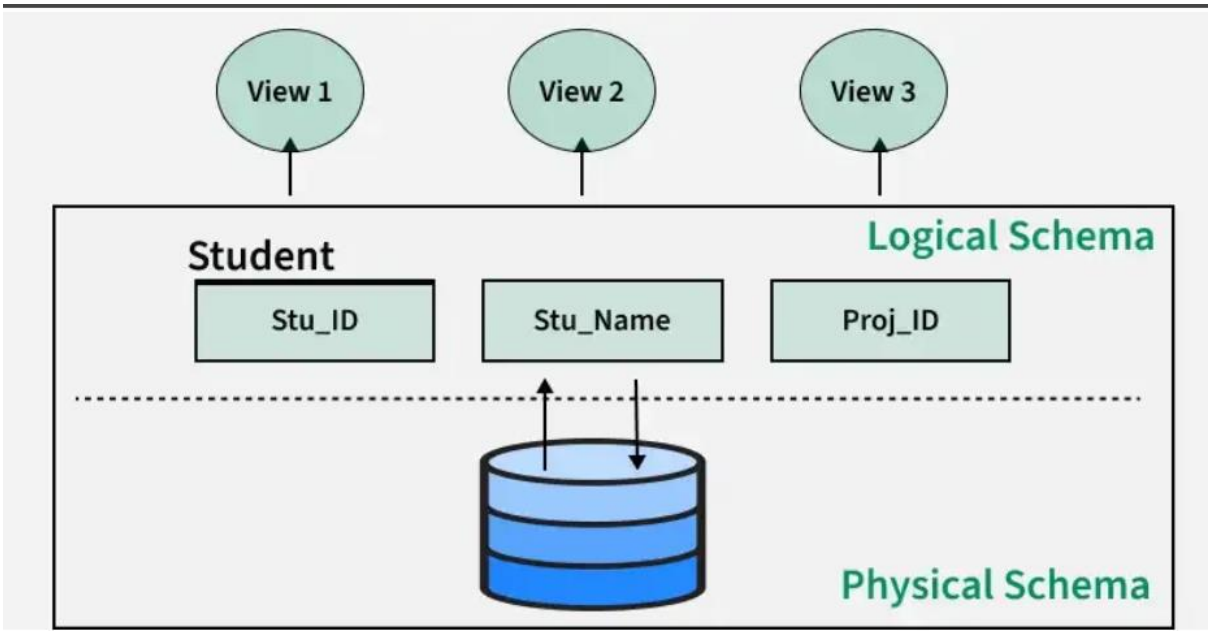
Database Schema

A database schema is the design or structure of a database that defines how data is organized and how different data elements relate to each other. It acts as a blueprint, outlining tables, fields, relationships, and rules that govern the data.

Key points about a database schema:

- It defines how data is logically organized, including tables, fields, and relationships.
- It outlines the relationships between entities, such as primary and foreign keys.
- It helps resolve issues with unstructured data by organizing it in a clear, structured way.
- Database schemas guide how data is accessed, modified, and maintained.

In simple terms, the schema provides the framework that makes it easier to understand, manage, and use data in a database. It's created by database designers to ensure the data is consistent and efficiently organized.



Types of Database Schemas

Types of Database Schemas

Conceptual Database Schema

A **conceptual schema** describes the overall structure of the entire database at a high level. It focuses on the *meaning* of data and captures how different entities relate to each other—without worrying about how data is stored physically.

Key Points:

- Represents the whole database for the entire organization.
- Describes entities, attributes, and relationships.
- Technology-independent (does not depend on SQL, storage, hardware, etc.).
- Usually designed using **ER diagrams**.
- Acts as a bridge between user requirements and the logical schema.

Example:

A conceptual schema for a school system includes entities such as:
Students, **Courses**, **Teachers**, **Departments**, and shows how they are related.

Physical Database Schema

- The **physical schema** defines **how the data is actually stored on disk**.

- Specifies **files, indexes, partitions, storage blocks, access paths**, etc.

It represents the **lowest level of abstraction**, focusing on performance, storage optimization, and data retrieval efficiency.

- Determines the physical layout of tables and indexes.
- Depends on the specific DBMS (e.g., MySQL, Oracle, PostgreSQL).
- Designed mainly by the **database administrator (DBA)**.

Logical Database Schema

- The **logical schema** describes the logical structure of data as it appears to the database designers and developers.
- Defines **tables, columns, primary keys, foreign keys, relationships, and integrity rules**.

It ensures:

- Data is structured properly
- Relationships between tables are meaningful
- Integrity rules maintain data accuracy during operations

This schema represents a **higher level of abstraction** and is independent of physical storage details.

- Built from the conceptual schema using ER modeling.
- Focuses on **how data is logically organized**, not how it is stored physically.

View Database Schema

- The **view schema** is the highest level of abstraction, showing how **individual users or applications see the database**.
- Defines **customized views** tailored to users' needs.

Users interact with these views without knowing about the underlying logical or physical structures.

Example:

- A student sees only their own marks and courses.
- An admin sees all student records.

Both are different view schemas derived from the same logical structure.

- A database can have **multiple view schemas**, each showing only part of the data.
- Helps ensure data security and simplicity by hiding unnecessary details.

Creating Database Schema

For creating a schema, the statement "CREATE SCHEMA" is used in every database. But different databases have different meanings for this. Below we'll be looking at some statements for creating a database schema in different database systems:

1. MySQL: In MySQL, we use the "CREATE SCHEMA" statement for creating the database, because, in MySQL CREATE SCHEMA and CREATE DATABASE, both statements are similar.

2. SQL Server: In SQL Server, we use the "CREATE SCHEMA" statement for creating a new schema.

3. Oracle Database: In Oracle Database, we use "CREATE USER" for creating a new schema, because in the Oracle database, a schema is already created with each database user. The statement "CREATE SCHEMA" does not create a schema, instead, it populates the schema with tables & views and also allows one to access those objects without needing multiple SQL statements for multiple transactions.

Database Schema Designs

There are many ways to structure a database and we should use the best-suited schema design for creating our database because ineffective schema designs are difficult to manage & consume extra memory and resources.

Schema design mostly depends on the application's requirements. Here we have some effective schema designs to create our applications, let’s take a look at the schema designs:

1. Flat Model
2. Hierarchical Model
3. Network Model
4. Relational Model
5. Star Schema
6. Snowflake Schema

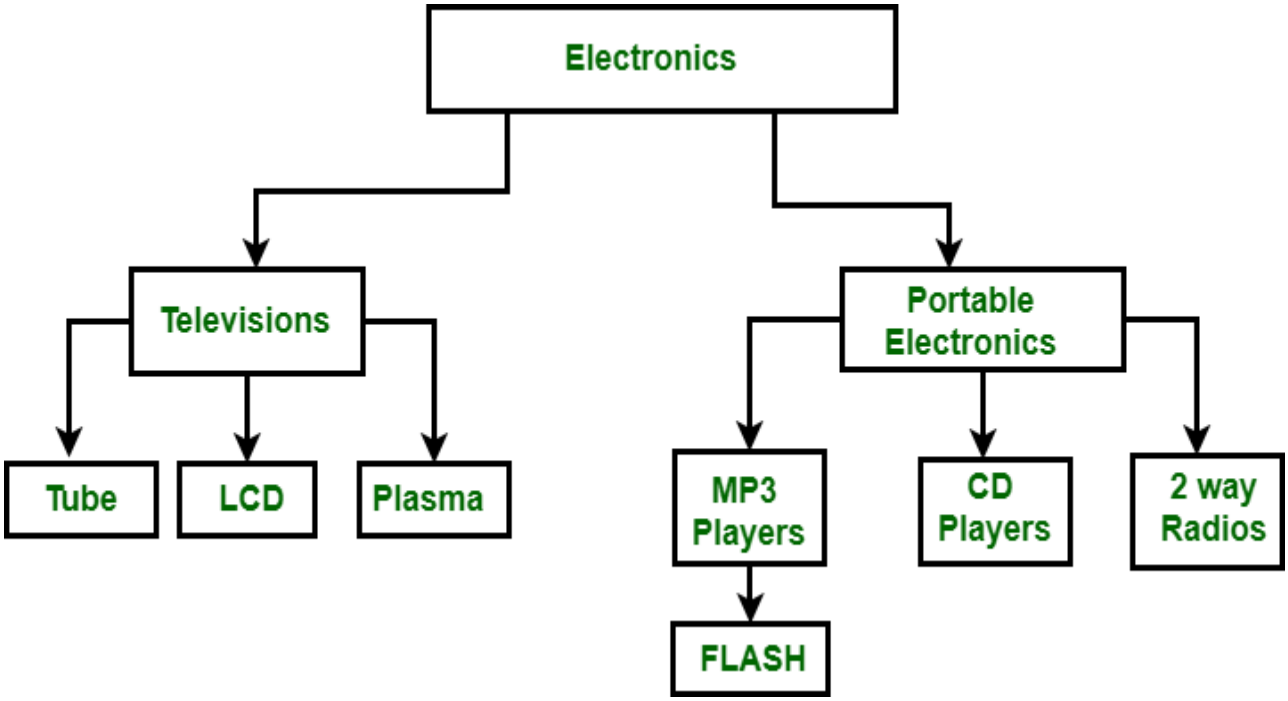
Flat Model

A flat model schema is a 2-D array in which every column contains the same type of data/information and the elements with rows are related to each other. It is just like a table or a spreadsheet. This schema is better for small applications that do not contain complex data.

Flat Model

Hierarchical Model

Data is arranged using parent-child relationships and a tree-like structure in the Hierarchical Database Model. Because each record consists of several children and one parent, it can be used to illustrate one-to-many relationships in diagrams such as organizational charts. A hierarchical database structure is great for storing nested data.

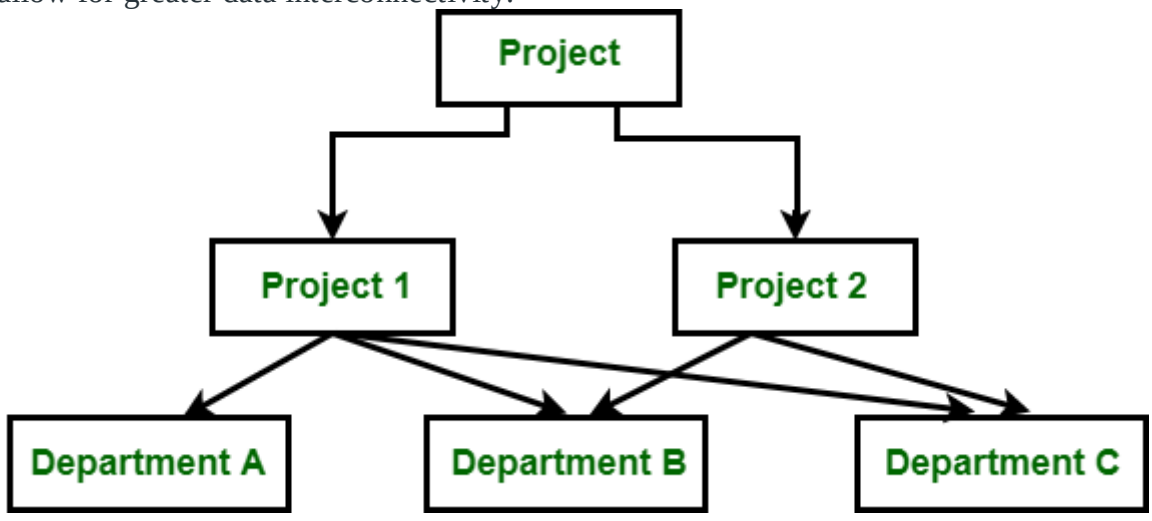


Designing Hierarchical Model

Network Model

The network model is similar to the hierarchical model in that it represents data using nodes (entities) and edges (relationships). However, unlike the hierarchical model, which enforces a strict parent-child relationship, the network model allows for more flexible many-to-many relationships. This flexibility means that a node can have multiple parent nodes and child nodes, making the structure more dynamic.

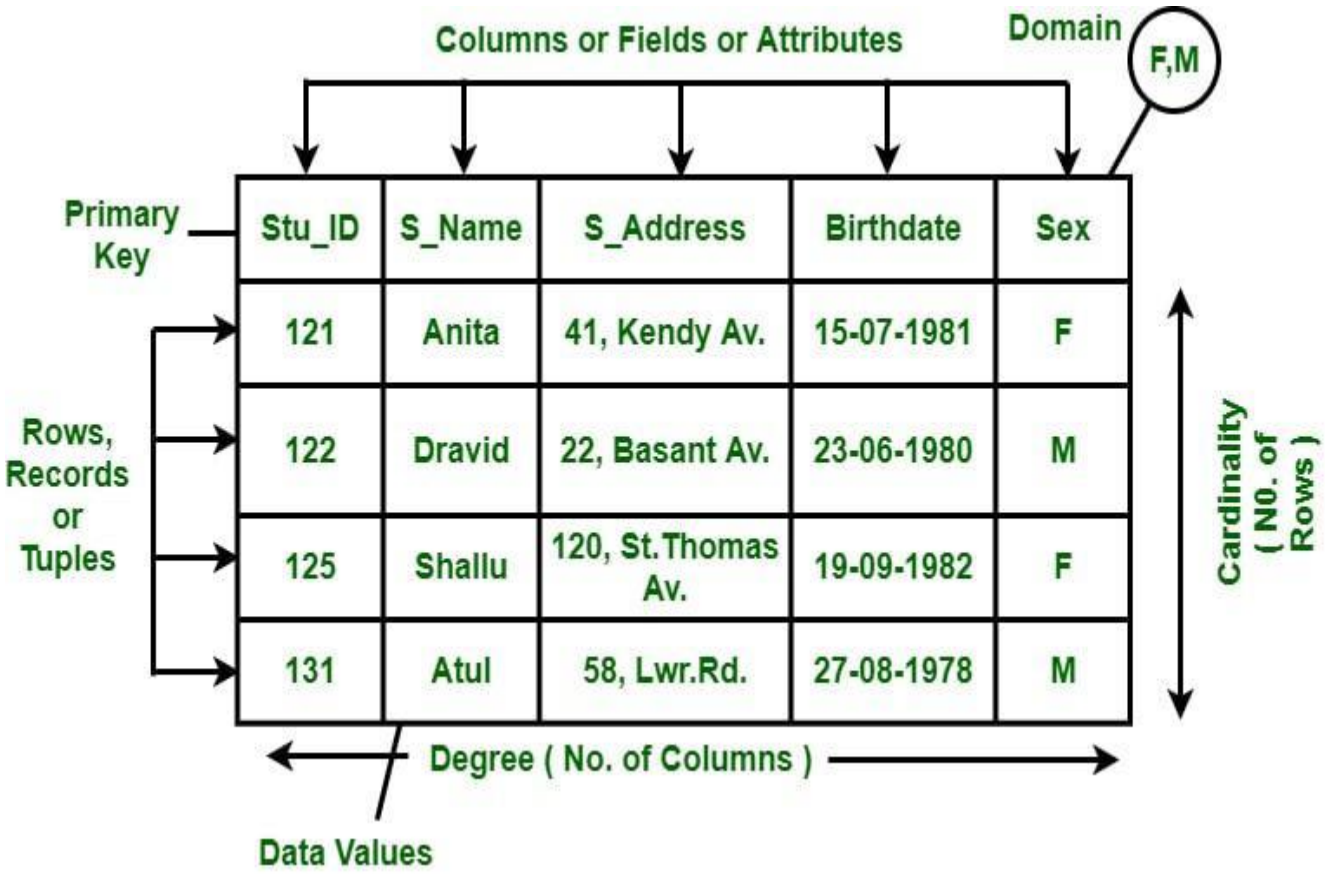
The network model can contain cycles which is a situation where a path exists that allows you to start and end at the same node. These cycles enable more complex relationships and allow for greater data interconnectivity.



Designing Network Model

Relational Model

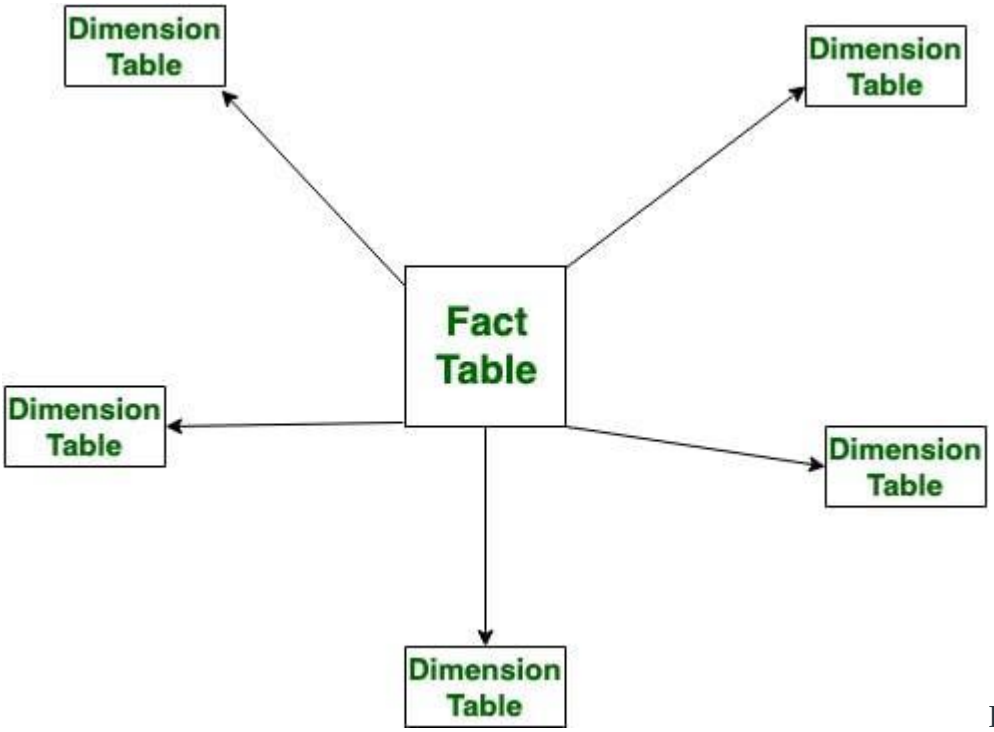
The relational model is mainly used for relational databases, where the data is stored as relations of the table. This relational model schema is better for object-oriented programming.



Designing Relational Model

Star Schema

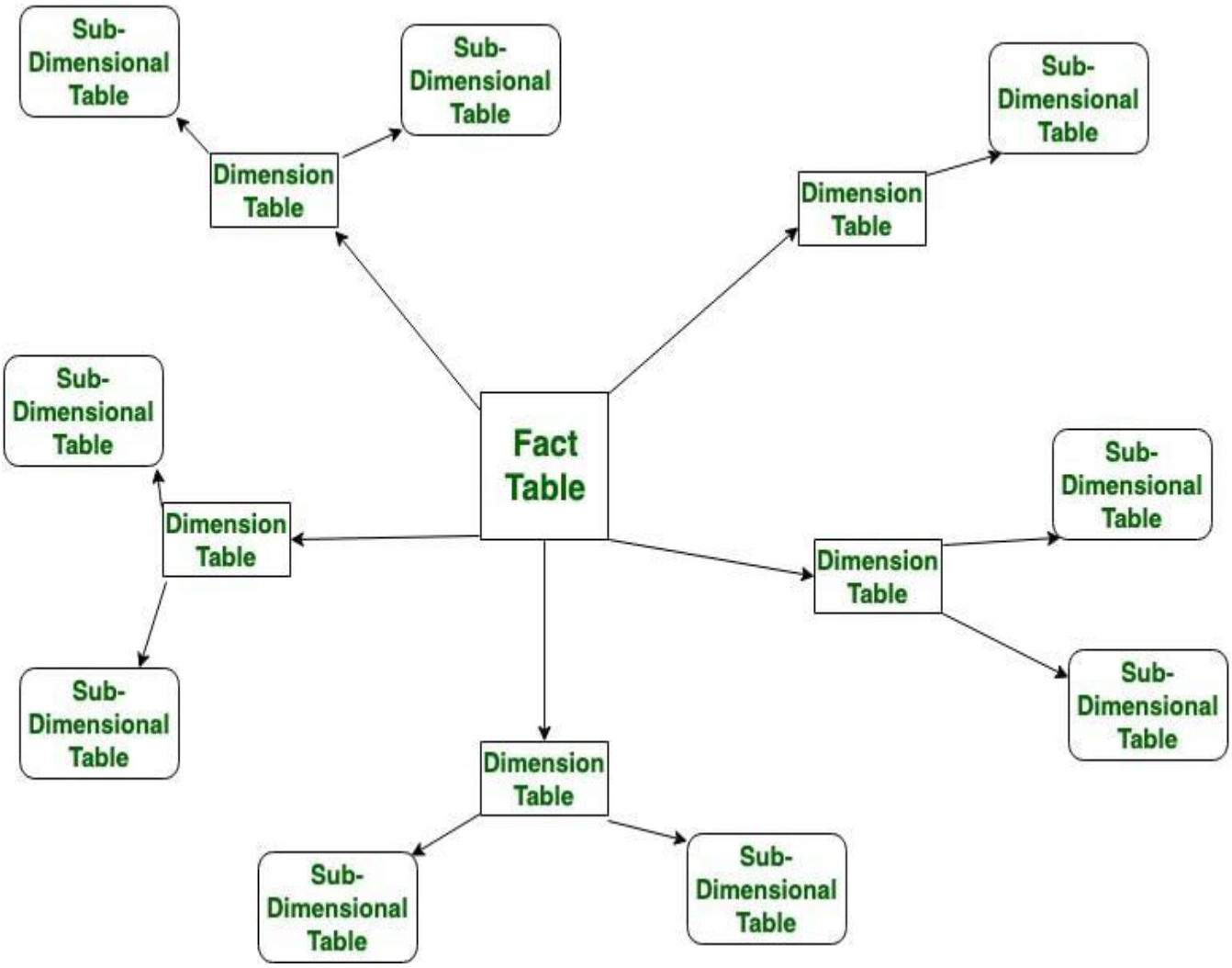
Star schema is better for storing and analyzing large amounts of data. It has a fact table at its center & multiple dimension tables connected to it just like a star, where the fact table contains the numerical data that run business processes and the dimension table contains data related to dimensions such as product, time, people, etc. or we can say, this table contains the description of the fact table. The star schema allows us to structure the data of RDBMS.



Designing Star Schema

Snowflake Schema

Just like star schema, the snowflake schema also has a fact table at its center and multiple dimension tables connected to it, but the main difference in both models is that in snowflake schema – dimension tables are further normalized into multiple related tables. The snowflake schema is used for analyzing large amounts of data.



Designing Snowflake Schema

Difference between Logical and Physical Database Schema

Physical Schema	Logical Schema
Physical schema describes the way of storage of data in the disk.	Logical schema provides the conceptual view that defines the relationship between the data entities.
Having Low level of abstraction.	Having a high level of abstraction.
The design of database is independent to any database management system.	The design of a database must work with a specific database management system or hardware platform.

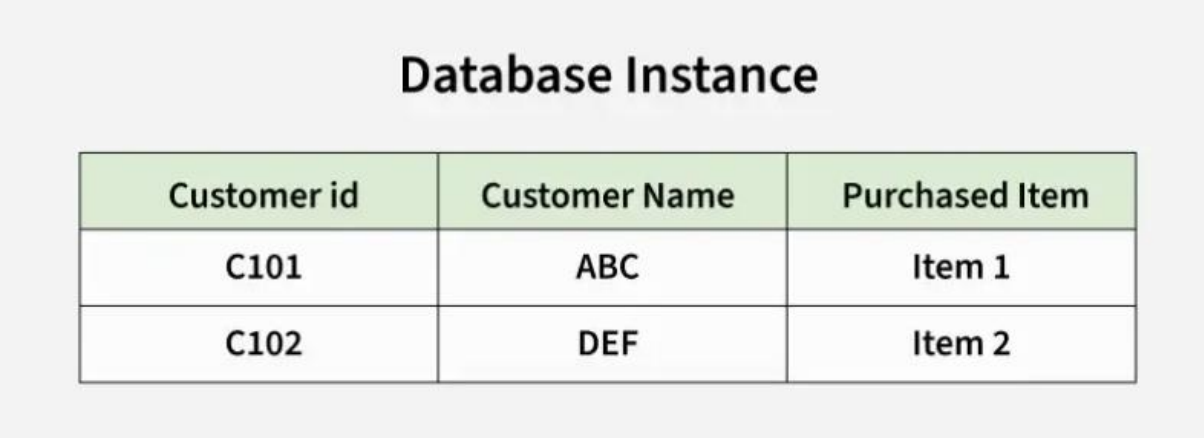
Physical Schema	Logical Schema
Changes in Physical schema effects the logical schema	Any changes made in logical schema have minimal effect in the physical schema
Physical schema does not include attributes.	Logical schema includes attributes.
Physical schema contains the attributes and their data types.	Logical schema does not contain any attributes or data types.
Examples: Data definition language(DDL), storage structures, indexes.	**Examples:** Entity Relationship diagram, Unified Modeling Language, class diagram.

Advantages of Database Schema

- **Providing Consistency of data:** Database schema ensures the data consistency and prevents the duplicates.
- **Maintaining Scalability:** Well designed database schema helps in maintaining addition of new tables in database along with that it helps in handling large amounts of data in growing tables.
- **Performance Improvement:** Database schema helps in faster data retrieval which is able to reduce operation time on the database tables.
- **Easy Maintenance:** Database schema helps in maintaining the entire database without affecting the rest of the database
- **Security of Data:** Database schema helps in storing the sensitive data and allows only authorized access to the database.

Database Instance

A database instance is a snapshot of a database at a specific moment in time, containing all the properties described by a database schema as data values. Unlike database schemas, which are considered the "blueprint" of a database, instances can change over time whereas it is very difficult to modify the schema because the schema represents the fundamental structure of the database. Database instance does not hold any information related to the saved data in database.



Instance

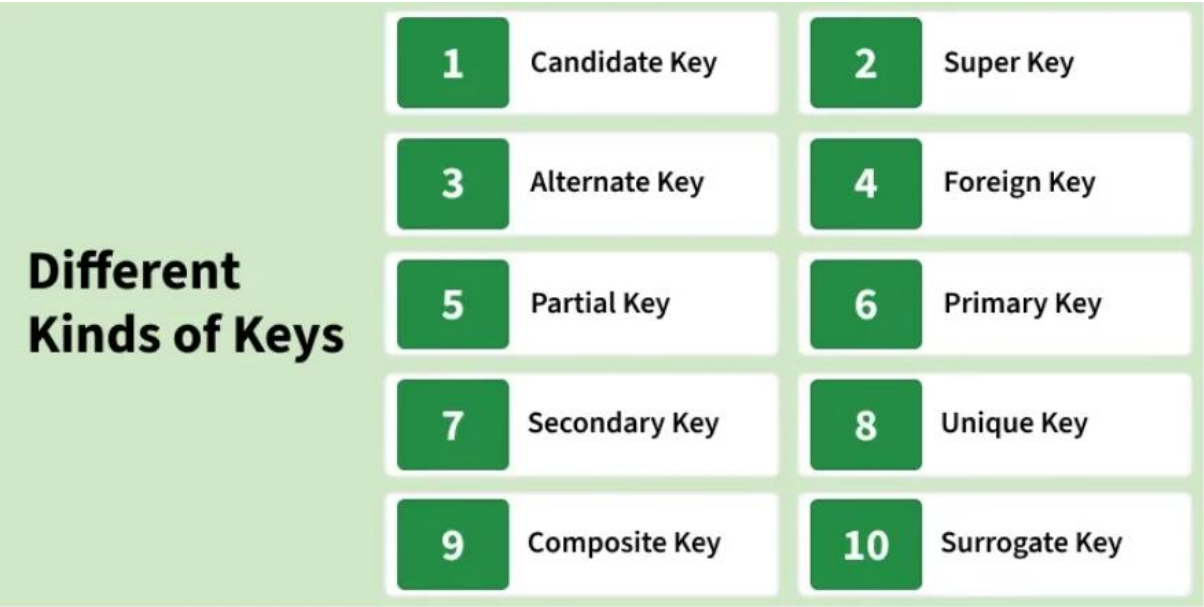
Database schema versus database instance

Aspect	Database Schema	Database Instance
Definition	Blueprint or design of the database structure	Actual data stored in the database at a given time
Nature	Static (does not change frequently)	Dynamic (changes with every data modification)
Represents	Structure (tables, columns, data types, relationships)	State of the data in the database
Example	Table definitions, data types, constraints	Actual rows of data in the tables
Change Frequency	Changes infrequently (e.g., during schema design changes)	Changes frequently with transactions

Keys in Relational Model

Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.

- They uniquely identify each row in a table.
- They prevent data duplication and maintain consistency.
- They create relationships between different tables.



Keys in DBMS

Importance of Keys in DBMS
Keys are important in a Database Management System (DBMS) for several reasons:

- **Uniqueness:** Keys ensure that each record in a table is unique and can be identified distinctly.
- **Data Integrity:** Keys prevent data duplication and maintain the consistency of the data.
- **Efficient Data Retrieval:** Keys help in creating relationships between tables, allowing faster queries and better data organization.

Without keys, managing large databases would become difficult, and data retrieval would be slow and error-prone.

Types of Database Keys

1. Super Key

The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as Super Key. It may include extra attributes that aren't important for uniqueness but still uniquely identify the row. For Example, STUD_NO, (STUD_NO, STUD_NAME), etc.

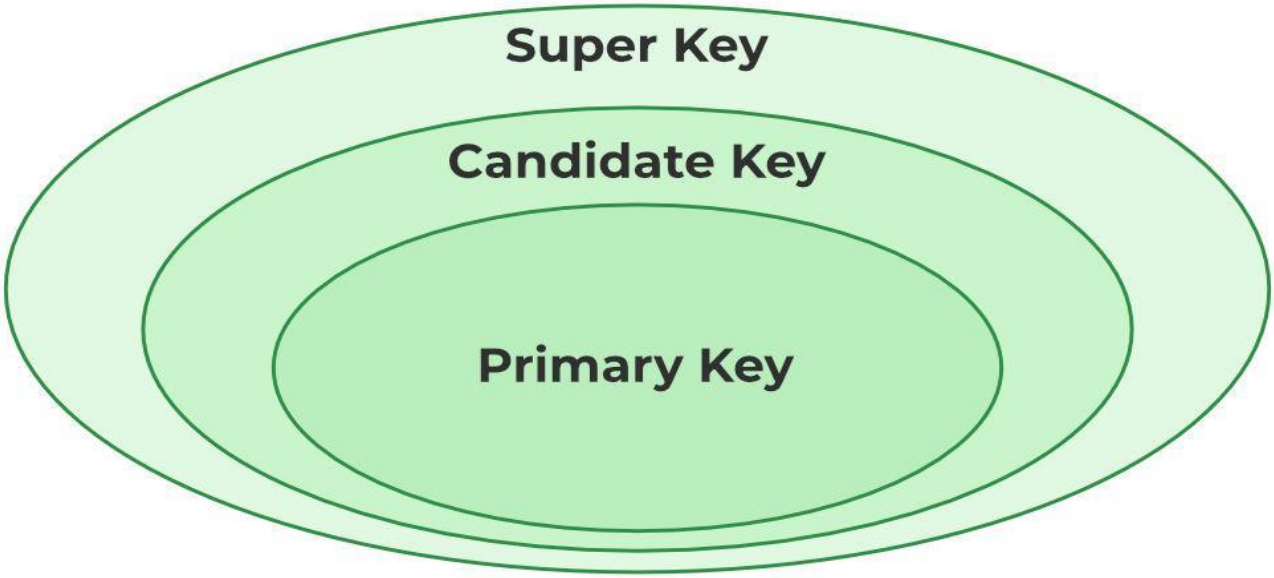
- A super key is a group of single or multiple keys that uniquely identifies rows in a table. It supports NULL values in rows.
- A super key can contain extra attributes that aren’t necessary for uniqueness.
- For example, if the "STUD_NO" column can uniquely identify a student, adding "SNAME" to it will still form a valid super key, though it's unnecessary.

Example: Consider the STUDENT table

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

A super key could be a combination of STUD_NO and PHONE, as this combination uniquely identifies a student.



Relation between Primary Key, Candidate Key, and Super Key

2. Candidate Key

The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

- A candidate key is a minimal super key, meaning it can uniquely identify a record but contains no extra attributes.
- It is a super key with no repeated data is called a candidate key.
- The minimal set of attributes that can uniquely identify a record.
- A candidate key must contain unique values, ensuring that no two rows have the same value in the candidate key's columns.
- Every table must have at least a single candidate key.
- A table can have multiple candidate keys but only one primary key.

Example: For the STUDENT table below, STUD_NO can be a candidate key, as it uniquely identifies each record.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

Table: STUDENT_COURSE

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

A composite candidate key example: {STUD_NO, COURSE_NO} can be a candidate key for a STUDENT_COURSE table.

3. Primary Key

A primary key is chosen from the set of candidate keys to uniquely identify each record in a table. For example, in the STUDENT table, both STUD_NO and STUD_PHONE can be candidate keys, but STUD_NO is selected as the primary key.

- It uniquely identifies every tuple (row) and does not allow duplicate values.
- It cannot be NULL, as each record must have a valid identifier.
- It may be single-column or composite (made of multiple columns).
- Databases often organize data using the primary key to allow faster access and searching.

Example: The STUDENT table has the structure Student(STUD_NO, SNAME, ADDRESS, PHONE), where STUD_NO is the primary key.

STUD_NO	SNAME	ADDRESS	PHONE
1	Tom	New York	123456789
2	John	Los Angeles	223365796
3	Alice	Chicago	175468965

STUDENT Table

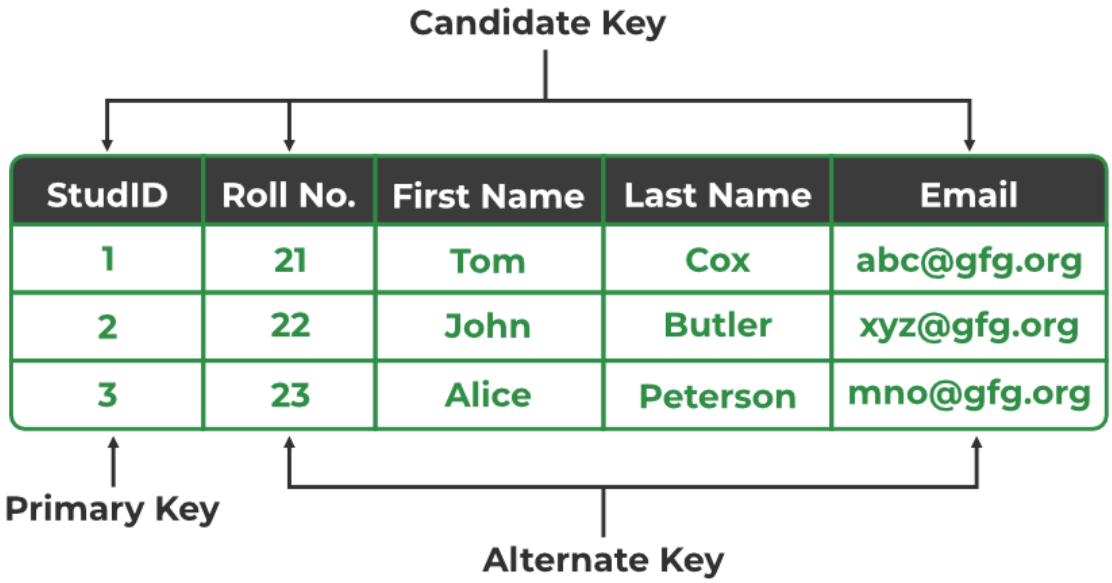
4. Alternate Key

An alternate key is any candidate key in a table that is not chosen as the primary key. In other words, all the keys that are not selected as the primary key are considered alternate keys.

- An alternate key is also referred to as a secondary key because it can uniquely identify records in a table, just like the primary key.

- An alternate key can consist of one or more columns (fields) that can uniquely identify a record, but it is not the primary key

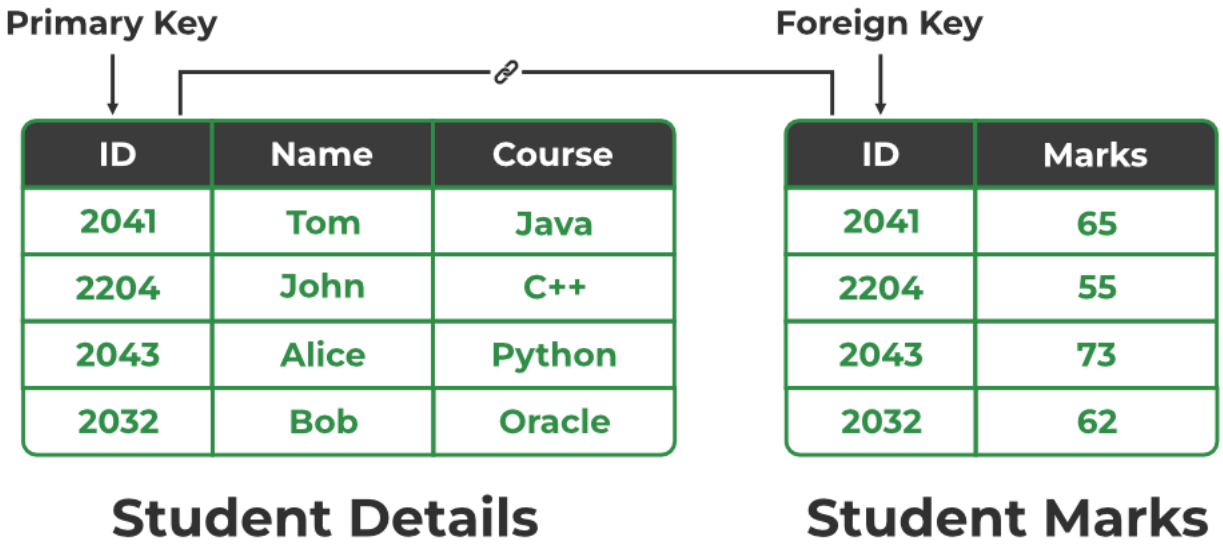
Example: In the STUDENT table, both STUD_NO and PHONE are candidate keys. If STUD_NO is chosen as the primary key, then PHONE would be considered an alternate key.



Primary Key, Candidate Key, and Alternate Key

5. Foreign Key

A foreign key is an attribute in one table that refers to the primary key in another table. The table that contains the foreign key is called the referencing table and the table that is referenced is called the referenced table.



Relation between Primary Key and Foreign Key

- A foreign key in one table points to the primary key in another table, establishing a relationship between them.
- It helps connect two or more tables, enabling you to create relationships between them. This is important for maintaining data integrity and preventing data redundancy.
- They act as a cross-reference between the tables.

Example: Consider the STUDENT_COURSE table

STUD_NO	TEACHER_NO	COURSE_NO
1	001	C001
2	056	C005

STUDENT_COURSE Table

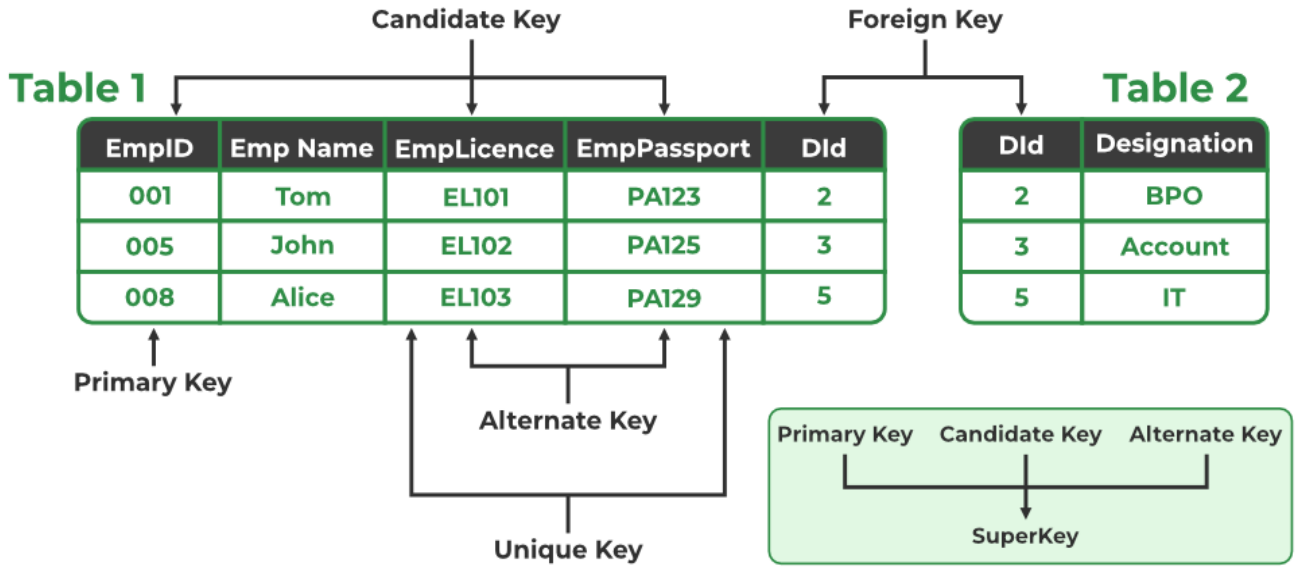
- STUD_NO in the STUDENT_COURSE table is a foreign key that refers to the STUD_NO primary key of the STUDENT table.
- Unlike a primary key, a foreign key can contain duplicate values and may be NULL. For example, STUD_NO appears multiple times in STUDENT_COURSE because a student can enroll in more than one course.
- However, STUD_NO in the STUDENT table is a primary key, so it must always be unique and non-NULL.

6. Composite Key

Sometimes, a single column is not enough to uniquely identify all records in a table, so a combination of multiple attributes is used. An optimal set of such attributes is chosen to ensure that every row is uniquely identifiable.

- It acts as a primary key if there is no primary key in a table
- Two or more attributes are used together to make a composite key .
- Different combinations of attributes may give different accuracy in terms of identifying the rows uniquely.

Example: In the STUDENT_COURSE table, {STUD_NO, COURSE_NO} can form a composite key to uniquely identify each record.



Set Operators in SQL

Set Operators in SQL are used to combine, compare or filter data by performing operations between two or more sets, enabling deeper and more flexible analysis. They allow users to analyze overlaps, differences and unions between data groups making it easier to answer complex business questions that go beyond simple filtering.

- Combines multiple sets using logical operations
- Supports Union, Intersection and Difference
- Helps compare overlapping and non-overlapping data
- Enables advanced segmentation and analysis

Create User Tables

Before exploring SQL set operators first create a simple dataset that represents users recorded across different years. Each table contains a UserID and Name, allowing us to compare users across time using set operations.

Table Structure

Each yearly table follows the same structure to ensure compatibility with SQL set operators.

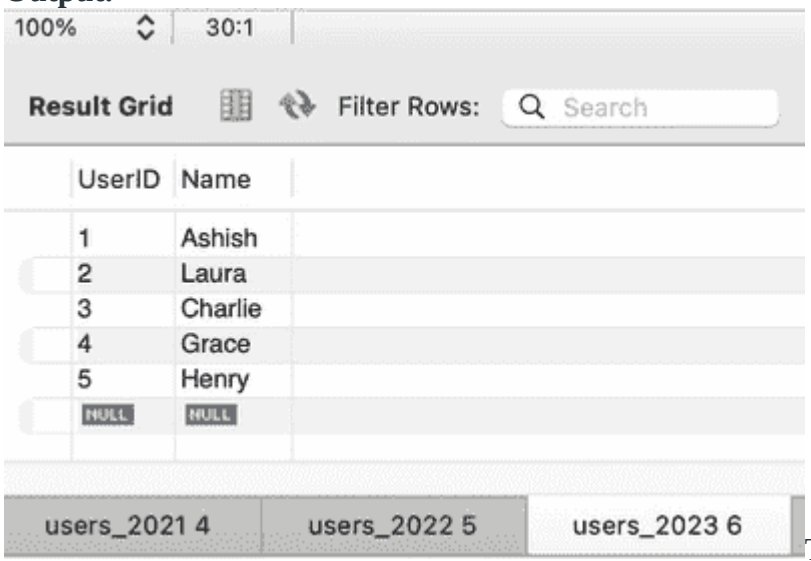
CREATE TABLE IF NOT EXISTS users.users_2021 (UserID INT PRIMARY KEY, Name VARCHAR(50));
CREATE TABLE IF NOT EXISTS users.users_2022 (UserID INT PRIMARY KEY, Name VARCHAR(50));
CREATE TABLE IF NOT EXISTS users.users_2023 (UserID INT PRIMARY KEY, Name VARCHAR(50));

Sample Data Insertion

We insert sample user records for each year to simulate new users joining and existing users continuing across years.

INSERT INTO users.users_2021 (UserID, Name) VALUES (1, 'Ashish'), (2, 'Laura'), (7, 'Prakash');
INSERT INTO users.users_2022 (UserID, Name) VALUES (1, 'Ashish'), (2, 'Laura'), (3, 'Charlie'), (4, 'Grace');
INSERT INTO users.users_2023 (UserID, Name) VALUES (1, 'Ashish'), (2, 'Laura'), (3, 'Charlie'), (4, 'Grace'), (5, 'Henry');

Output:



Table

Types of Set Operators in SQL

1. UNION

Combines the result sets of two or more SELECT statements and removes duplicate rows.

Syntax:

```
SELECT columns FROM table1
UNION
SELECT columns FROM table2;
```

Example: Union removes duplicates and we are using a similar database of users which we have created previously in joins.

```
SELECT * FROM users.users_2021
UNION
SELECT * FROM users.users_2022;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
3	Charlie
4	Grace

query output

```
SELECT * FROM users.users_2021
UNION
SELECT * FROM users.users_2023;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
3	Charlie
4	Grace
5	Henry

query output

2. UNION ALL

Combines the result sets of two or more SELECT statements without removing duplicates.

Syntax:

```
SELECT columns FROM table1
UNION ALL
SELECT columns FROM table2;
```

Example:

```
SELECT * FROM users.users_2021
UNION ALL
SELECT * FROM users.users_2022;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
1	Ashish
2	Laura
3	Charlie
4	Grace
5	Henry

query output

3. EXCEPT (selects the record which is not present in second table)

EXCEPT **is used to** find records that are present in the first table but NOT present in the second table.

Jo data **first table me hai**
☞ But **second table me nahi hai**

Wo result me aayega

Table A (2021)

Riya
Rahul
Amit

Table B (2022)

Riya
Rahul

Query:

A EXCEPT B

👉 Output:
Amit ✅

Returns the rows from the first SELECT statement that are not in the second SELECT statement.

Syntax:

```
SELECT columns FROM table1
EXCEPT
SELECT columns FROM table2;
```

Example:

```
SELECT * FROM users.users_2021
EXCEPT
SELECT * FROM users.users_2022;
```

UserID	Name
7	Prakash

query output

```
SELECT * FROM users.users_2023
EXCEPT
SELECT * FROM users.users_2021;
```

UserID	Name
3	Charlie
4	Grace
5	Henry

query output

4. INTERSECT

Returns only the rows that are common to both SELECT statements.

Syntax:

```
SELECT columns FROM table1
INTERSECT
SELECT columns FROM table2;
```

Example:

```
SELECT * FROM users.users_2021
INTERSECT
SELECT * FROM users.users_2022;
```

UserID	Name
1	Ashish
2	Laura

query output

```
SELECT * FROM users.users_2022
INTERSECT
SELECT * FROM users.users_2023;
```

UserID	Name
1	Ashish
2	Laura
3	Charlie
4	Grace

query output

Combining Multiple Set Operators

We can combine multiple set operators to create complex queries.

***Note**: UNION and UNION ALL can be combined, but the use of parentheses ensures the correct order of operations when combining multiple operators.*

```
SELECT * FROM users.users_2021
UNION ALL
SELECT * FROM users.users_2022
UNION
SELECT * FROM users.users_2023;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
1	Ashish
2	Laura
3	Charlie
4	Grace
1	Ashish
2	Laura
3	Charlie
4	Grace
5	Henry

query output

1. List the New Users Added in 2022

Identifies users who appeared in 2022 but were not present in 2021.

```
SELECT * FROM users.users_2022
EXCEPT
SELECT * FROM users.users_2021;
```

UserID	Name
3	Charlie
4	Grace

query output

2. List the new users added in 2023

Finds users who joined in 2023 and did not exist in the 2022 dataset.

```
SELECT * from users.users_2023
EXCEPT
SELECT * from users.users_2022;
```

UserID	Name
5	Henry

query output

3. List the users who left us

Returns users who were present in 2021 but are missing in both 2022 and 2023.

```
SELECT * FROM users.users_2021
EXCEPT
SELECT * FROM users.users_2022
EXCEPT
SELECT * FROM users.users_2023;
```

UserID	Name
7	Prakash

query output

4. List All the Users That Were There Throughout the Database for 2021 & 2022

Lists all users who existed at any point during 2021 or 2022.

```
SELECT * FROM users.users_2022
UNION
SELECT * FROM users.users_2021;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
3	Charlie
4	Grace

query output

5. All Users Across All Years

Combines all unique users who appeared in 2021, 2022 or 2023.

```
SELECT * FROM users.users_2021
UNION
SELECT * FROM users.users_2022
UNION
SELECT * FROM users.users_2023;
```

UserID	Name
1	Ashish
2	Laura
7	Prakash
3	Charlie
4	Grace
5	Henry

query output

6. List the Users That Are There With Us for the Last 3 Years

Identifies users who consistently appear in all three yearly datasets.

```
SELECT * FROM users.users_2021
INTERSECT
SELECT * FROM users.users_2022
INTERSECT
SELECT * FROM users.users_2023;
```

UserID	Name
1	Ashish
2	Laura

query output

7. List All Users Except Those Who Are There With Us for 3 Years

Returns all users except those who have been present continuously for all three years.

```
SELECT * FROM users.users_2021
UNION
SELECT * FROM users.users_2022
UNION
SELECT * FROM users.users_2023
EXCEPT
SELECT * FROM users.users_2021
INTERSECT
SELECT * FROM users.users_2022
INTERSECT
SELECT * FROM users.users_2023;
```

UserID	Name
7	Prakash
3	Charlie
4	Grace
5	Henry

SQL Aggregate functions

SQL Aggregate Functions allow summarizing large sets of data into meaningful results, making it easier to analyze patterns and trends across many records. They return a single output value after processing multiple rows in a table.

- Perform calculations like totals, averages, minimum or maximum values on data.
- Ignore NULL values in most functions except COUNT(*), improving result accuracy.
- Work with clauses such as GROUP BY, HAVING and ORDER BY for analysis.

Example: First, we create a demo SQL database and table, on which we use the Aggregate functions.

EmpID	Name	Salary
1	John	5000
2	Sara	
3	Michael	8000
4	Emma	5000

Employee Table

Query:

```
SELECT SUM(Salary) FROM Employee;
```

Output:

SUM(Salary)
18000

Syntax:

```
AGGREGATE_FUNCTION(column_name)
```

Aggregate Functions in SQL

Below are the most frequently used aggregate functions in SQL.

1. Count()

It is used to count the number of rows in a table. It helps summarize data by giving the total number of entries. It can be used in different ways depending on what you want to count:

- **COUNT(*):** Counts all rows.
- **COUNT(column_name):** Counts non-NULL values in the specified column.
- **COUNT(DISTINCT column_name):** Counts unique non-NULL values in the column.

Query:

```
-- Total number of records in the table
SELECT COUNT(*) AS TotalRecords FROM Employee;

-- Count of non-NULL salaries
SELECT COUNT(Salary) AS NonNullSalaries FROM Employee;
```

```
-- Count of unique non-NULL salaries
SELECT COUNT(DISTINCT Salary) AS UniqueSalaries FROM Employee;
```

Output:

TotalRecords
4

NonNullSalaries
3

UniqueSalaries
2

- COUNT(*) returns the total number of rows in the table, including rows with NULL values.
- COUNT(Salary) counts only the rows where Salary is not NULL.
- COUNT(DISTINCT Salary) counts unique non-NULL salary values, ignoring duplicates.

2. SUM()

It is used to calculate the total of a numeric column. It adds up all non-NULL values in that column for Example, SUM(column_name) returns sum of all non-NULL values in the specified column.

Query:

```
-- Calculate the total salary
SELECT SUM(Salary) AS TotalSalary FROM Employee;

-- Calculate the sum of unique salaries
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum FROM Employee;
```

Output:

TotalSalary
18000

DistinctSalarySum
13000

- SUM(Salary) adds all non-NULL salary values to get the total salary amount.
- SUM(DISTINCT Salary) adds only unique non-NULL salary values, avoiding duplicates.
- NULL values are ignored in both SUM calculations.

3. AVG()

It is used to calculate average value of a numeric column. It divides sum of all non-NULL values by the number of non-NULL rows for Example, AVG(column_name) returns average of all non-NULL values in the specified column.

Query:

```
-- Calculate the average salary
SELECT AVG(Salary) AS AverageSalary FROM Employee;

-- Average of distinct salaries
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary FROM Employee;
```

Output:

AverageSalary
6000

DistinctAvgSalary
6500

- AVG(Salary) calculates the average of all non-NULL salary values.
- AVG(DISTINCT Salary) computes the average only from unique non-NULL salary values.
- Both ignore NULL values when performing the calculation.

4. MIN() and MAX()

The MIN() and MAX() functions return the smallest and largest values, respectively, from a column.

Query:

```
-- Find the highest salary
SELECT MAX(Salary) AS HighestSalary FROM Employee;

-- Find the lowest salary
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Output:

HighestSalary
8000

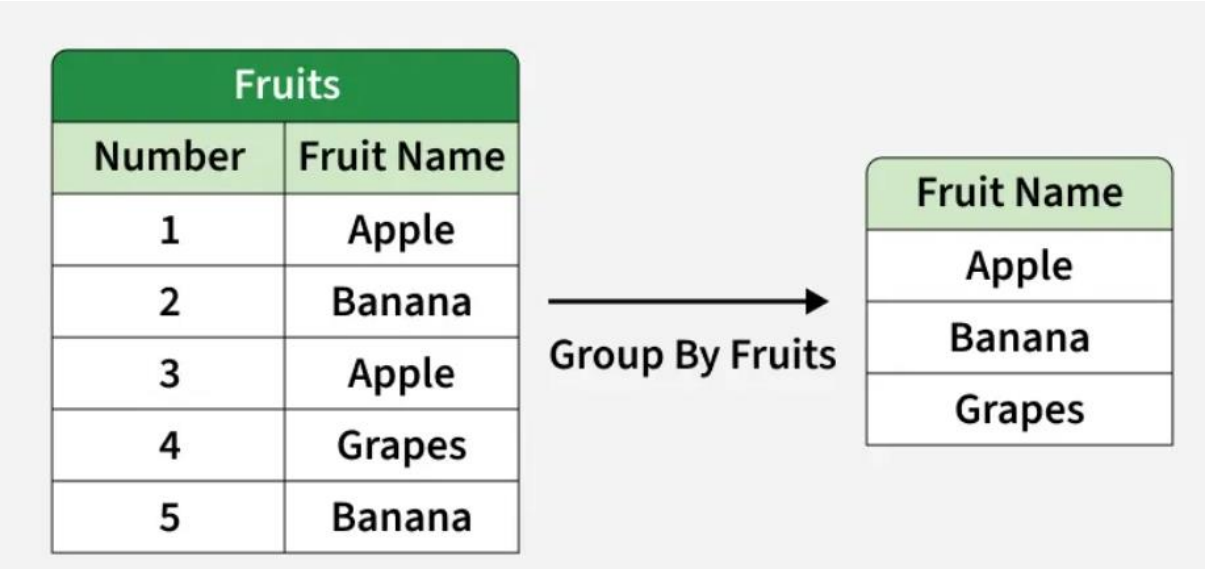
LowestSalary
5000

- MAX(Salary) returns the highest non-NULL salary value from the Employee table.

- MIN(Salary) returns the lowest non-NULL salary value from the Employee table.
- Both functions ignore NULL values while determining the result.

SQL GROUP BY

The SQL GROUP BY clause is used to arrange identical data into groups based on one or more columns. It is commonly used with aggregate functions like COUNT(), SUM(), AVG(), MAX() and MIN() to perform calculations on each group of data.



Example: First, we will create a demo SQL database and table, on which we will use the GROUP BY command.

EmpID	Name	Department	Salary
1	John Smith	HR	45000
2	Emma Brown	IT	60000
3	David Lee	Sales	52000
4	James Brown	IT	58000

Employees Table

Query:

```
SELECT Department, SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department;
```

Output:

Department	TotalSalary
HR	45000
IT	118000
Sales	52000

- Groups all employee records based on their department.
- Calculates and returns total salary for each department.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1, column2;
```

- **aggregate_function:** function used for aggregation, e.g., SUM(), AVG(), COUNT().
- **table_name:** name of the table from which data is selected.
- **condition:** Optional condition to filter rows before grouping (used with WHERE).
- **column1, column2:** Columns on which the grouping is applied.

Working with GROUP BY

Let's assume that we have a Student table. We will insert some sample data into this table and then perform operations using GROUP BY to understand how it groups rows based on a column and aggregates data.

name	year	subject
Avery	1	Mathematics
Elijah	2	English
Harper	3	Science
James	1	Mathematics
Charlotte	2	English
Benjamin	3	Science

Student Table

Example 1: Group By Single Column

When we group by a single column, rows with the same value in that column are combined. For example, grouping by subject shows how many students are enrolled in each subject.

Query:

```
SELECT subject, COUNT(*) AS Student_Count
FROM Student
GROUP BY subject;
```

Output:

subject	Student_Count
English	2
Mathematics	2
Science	2

- Groups all student records by their subject.
- Counts total number of students in each subject.

Example 2: Group By Multiple Columns

Using GROUP BY with multiple columns groups rows that share the same values in those columns. For example, grouping by subject and year will combine rows with the same subject–year pair and we can count how many students fall into each group.

Query:

```
SELECT subject, year, COUNT(*)
FROM Student
GROUP BY subject, year;
```

Output:

subject	year	COUNT(*)
English	2	2
Mathematics	1	2
Science	3	2

- Students with the same subject and year are grouped together.
- Since each subject and year pair occurs twice, the count is 2 for every group.

HAVING Clause in GROUP BY Clause

HAVING clause is used to filter results after grouping, especially when working with aggregate functions like SUM(), COUNT() or AVG(). Unlike WHERE, it applies conditions on grouped data.

emp_no	name	sal	age
1	Liam	50000	25
2	Emma	60000.5	30
3	Noah	75000.75	35
4	Olivia	45000.25	28
5	Ethan	80000	32
6	Sophia	65000	27
7	Mason	55000.5	29

Employees Table

Example 1: Filter by Total Salary

In this query, we group employees by age and display only those whose total salary is greater than 50,000.

```
SELECT age, SUM(sal) FROM Employees
GROUP BY age
HAVING SUM(sal)>50000;
```

Output:

age	SUM(sal)
27	65000
29	55000.5
30	60000.5
32	80000
35	75000.75

- Groups employees by age and sums their salaries.
- Returns only ages where total salary exceeds 50000.

Example 2: Filter by Average Salary

In this query, we group employees by age and display only those age groups where average salary is above 60,000.

```
SELECT age, AVG(sal) AS Average_Salary
FROM Employees
```

```
GROUP BY age
HAVING AVG(sal) > 60000;
```

Output:

age	Average_Salary
27	65000
30	60000.5
32	80000
35	75000.75

- Groups employees by age and calculates average salary.
- Filters only age groups with average salary above 60000.

SQL Subquery

A subquery in SQL is a query nested inside another SQL query. It allows complex filtering, aggregation and data manipulation by using the result of one query inside another. They are an essential tool when we need to perform operations like:

- Filter rows based on results from another query.
- Apply aggregate functions like SUM, COUNT, or AVG dynamically.
- Update data using values from other tables.
- Delete rows based on conditions returned by another query.

Example: First, we create a demo SQL database and tables, on which we use the SQL Subqueries.

StudentID	Name	Score
1	Alice	88
2	Daniel	92
3	Maria	79

Query:

```
SELECT * FROM Students
WHERE Score >  ( SELECT AVG(Score) FROM Students );
```

Output:

StudentID	Name	Score
1	Alice	88
2	Daniel	92

- The subquery calculates the average score of all students.
- The main query returns only the students whose score is higher than that average.

Syntax:

```
SELECT column_name
FROM table_name
```

```
WHERE column_name expression operator
  (SELECT column_name FROM table_name WHERE ...);
```

Note: *Subqueries do not have a single fixed syntax, as they can be used in different clauses like SELECT, WHERE, FROM and HAVING*

SQL Clauses for Subqueries
Clauses that can be used with subqueries are:

- Filters rows in the outer query based on the results returned by the WHERE clause.
- Treats the subquery as a temporary (derived) table that can be queried like a normal table using the FROM clause.
- Filters grouped or aggregated results using values produced by the HAVING clause.

Types of Subqueries
Consider the following two tables for examples:

EmployeeID	Name	Salary	DepartmentID
1	Alice	80000	10
2	Bob	60000	10
3	Charlie	90000	20
4	Diana	75000	20
5	Evan	50000	30

Employees Table

DepartmentID	DepartmentName	Location
10	HR	New York
20	Engineering	Chicago
30	Sales	New York

Departments Table

1. Single-Row Subquery

A single-row subquery is a subquery that returns only one value.

- Returns exactly one row as the result.
- Commonly used with comparison operators such as =, >, <

Example:

```
SELECT * FROM Employees
WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

Output:

EmployeeID	Name	Salary	DepartmentID
3	Charlie	90000	20

- Finds the highest salary in the Employees table.
- Returns the employee(s) whose salary matches that maximum value.

2. Multi-Row Subquery

A multi-row subquery is a subquery that returns more than one value.

- Returns multiple rows as the result.
- Requires operators that can handle multiple values, such as IN, ANY or ALL

Example:

```
SELECT * FROM Employees
WHERE DepartmentID IN (SELECT DepartmentID FROM Departments WHERE
Location = 'New York');
```

Output:

EmployeeID	Name	Salary	DepartmentID
1	Alice	80000	10
2	Bob	60000	10
5	Evan	50000	30

- Identifies all departments located in New York.
- Returns the employees who belong to any of those departments.

3. Correlated Subquery

A correlated subquery is a subquery that depends on the outer query for its values.

- A dependent subquery: it references columns from the outer query.
- Executed once for each row of the outer query, making it slower for large datasets.

Example:

```
SELECT e.Name, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
                  FROM Employees
                  WHERE DepartmentID = e.DepartmentID);
```

Output:

Name	Salary
Alice	80000
Charlie	90000

- Calculates the average salary within each employee's department.
- Returns employees whose salary is higher than their department’s average.

Examples of Using SQL Subqueries

These examples showcase how subqueries can be used for various operations like selecting, updating, deleting or inserting data, providing insights into their syntax and functionality. Through these examples, we will understand flexibility and importance of subqueries in simplifying complex database tasks.

Consider the following two tables:

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
James	101	New York	9876543210
Sophia	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Oliver	105	Berlin	5432109876

Student_Info

NAME	ROLL_NO	SECTION
Emma	104	A
Oliver	105	B
Sophia	102	A

Student_Section

Example 1: Fetching Data Using Subquery in WHERE Clause
This example demonstrates how to use a subquery inside the WHERE clause. The inner query retrieves roll numbers of students who belong to section 'A' and the outer query fetches their corresponding details (name, location and phone number) from the Student table.

Query:

```
SELECT NAME, LOCATION, PHONE_NUMBER
FROM Student_Info
WHERE ROLL_NO IN (
   SELECT ROLL_NO
   FROM Student_Section
   WHERE SECTION = 'A'
);
```

Output:

NAME	LOCATION	PHONE_NUMBER
Sophia	London	8765432109
Emma	Sydney	6543210987

- The subquery NAME, LOCATION and PHONE_NUMBER, WHERE SECTION = 'A' finds roll numbers of students in section A.
- The outer query then uses these roll numbers to fetch details from the Student table.
- Thus, only Sophia and Emma are returned, since they are in section A.

Example 2: Using Subquery with DELETE
In this example, we use a subquery with DELETE to remove certain rows from the Student table. Instead of hardcoding roll numbers, the subquery finds them based on conditions.

Query:

```
DELETE FROM Student_Info
WHERE ROLL_NO IN (
   SELECT ROLL_NO FROM (
      SELECT ROLL_NO FROM Student_Info WHERE ROLL_NO <= 101 OR
ROLL_NO = 201
   ) AS temp
);
```

Output:

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
Sophia	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Oliver	105	Berlin	5432109876

- The subquery selects roll numbers 101 and 201.
- The outer query deletes students having those roll numbers.

Example 3: Using Subquery with UPDATE
Subqueries can also be used with UPDATE. In this example, we update student names to "Geeks" if their location matches the result of a subquery.

Query:

```
UPDATE Student_Info
SET NAME = 'Geeks'
WHERE LOCATION IN (
   SELECT LOCATION
   FROM Student_Info
   WHERE LOCATION IN ('London', 'Berlin')
);
```

Output:

NAME	ROLL_NO	LOCATION	PHONE_NUMBER
Geeks	102	London	8765432109
Daniel	103	Toronto	7654321098
Emma	104	Sydney	6543210987
Geeks	105	Berlin	5432109876

- The subquery selects locations 'London' and 'Berlin'.
- The outer query updates the NAME field for students whose location matches those values.

Example 4: Simple Subquery in the FROM Clause
This example demonstrates using a subquery inside the FROM clause, where the subquery acts as a temporary (derived) table.

Query:

```
SELECT NAME, PHONE_NUMBER
FROM (
   SELECT NAME, PHONE_NUMBER, LOCATION
   FROM Student_Info
   WHERE LOCATION LIKE 'T%'
) AS subquery_table;
```

Output:

NAME	PHONE_NUMBER
Daniel	7654321098

- The subquery (SELECT NAME, PHONE_NUMBER, LOCATION FROM Student WHERE LOCATION LIKE 'T%') fetches students whose location starts with "T" (Toronto).
- The outer query then selects only NAME and PHONE_NUMBER from this derived table.

Example 5: Subquery with JOIN
We can also use subqueries along with JOIN to connect data across tables.

Query:

```
SELECT s.NAME, s.LOCATION, ns.SECTION
FROM Student_Info s
INNER JOIN (
   SELECT ROLL_NO, SECTION
   FROM Student_Section
   WHERE SECTION = 'A'
) ns
ON s.ROLL_NO = ns.ROLL_NO;
```

Output:

NAME	LOCATION	SECTION
Emma	Sydney	A
Geeks	London	A

- The subquery extracts roll numbers of students in section A.
- Joining this with the Student_Info table on ROLL_NO returns Emma and Sophia along with their locations and section.

Joins in DBMS

Join is an operation in DBMS(Database Management System) that combines the rows of two or more tables based on related columns between them. The main purpose of join is to retrieve the data from multiple tables in other words Join is used to perform multi-table queries. It is denoted by ⋈.

Syntax

$R3 <- \bowtie^{(R1)} <join\_condition> ^{(R2)}$

where R1 and R2 are two relations to be joined and R3 is a relation that will hold the result of the join operation.

Example

$$Temp <- \bowtie^{(student)}_{S.roll=E.roll}{}^{(Exam)}$$

where S and E are aliases of the student and exam respectively.

JOIN Example

Consider the two tables below as follows:

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	HARSH	DELHI	XXXXXXXXXX	18
2	PRATIK	BIHAR	XXXXXXXXXX	19
3	RIYANKA	SILIGURI	XXXXXXXXXX	20
4	DEEP	RAMNAGAR	XXXXXXXXXX	18
5	SAPTARHI	KOLKATA	XXXXXXXXXX	19
6	DHANRAJ	BARABAJAR	XXXXXXXXXX	20
7	ROHIT	BALURGHAT	XXXXXXXXXX	18
8	NIRAJ	ALIPUR	XXXXXXXXXX	19

Table 1 - Student

COURSE_ID	ROLL_NO
1	1
2	2
2	3
3	4
1	5
4	9
5	10
4	11

Table 2 - Student_Course

Both these tables are connected by one common key (column) i.e. ROLL_NO.

We can perform a JOIN operation using the given relational algebra:

$Student \bowtie Student\_course$

Output:

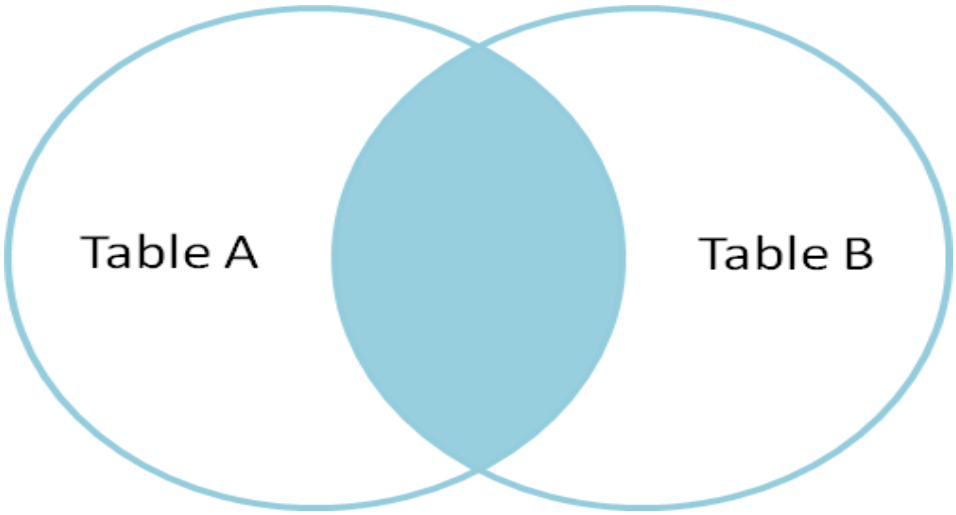
ROLL_NO	NAME	ADDRESS	PHONE	AGE	COURSE_ID
1	HARSH	DELHI	xxxxxxxxxx	18	1
2	PRATIK	BIHAR	xxxxxxxxxx	19	2
3	PRIYANKA	SILIGURI	xxxxxxxxxx	20	2
4	DEEP	RAMNAGAR	xxxxxxxxxx	18	3
5	SAPTARHI	KOLKATA	xxxxxxxxxx	19	1

Types of Join

There are many types of Joins in SQL. Depending on the use case, you can use different types of SQL JOIN clauses. Here are the frequently used SQL JOIN types:

1. Inner Join

Inner Join is a join operation in DBMS that combines two or more tables based on related columns and returns only rows that have matching values among tables. Inner join has two types.



Inner Join

- Conditional join
- Equi Join

- Natural Join

(a) Conditional Join

Conditional join or Theta join is a type of inner join in which tables are combined based on the specified condition.

In conditional join, the join condition can include <, >, <=, >=, ≠ operators in addition to the '=' operator.

Example: Suppose two tables A and B

Table A

R	S
10	5
7	20

Table B

T	U
10	12
17	6

$A \bowtie_{S<T} B$

Output

R	S	T	U
10	5	10	12
10	5	17	6

Explanation: This query joins the table A, B and projects attributes R, S, T, U were the condition S < T is satisfied.

(b) Equi Join

Equi Join is a type of inner join where the join condition uses the equality operator ('=') between columns.

Example: Suppose there are two tables Table A and Table C
Table A

Column A	Column B
a	a
a	b

Table C

Column A	Column B
a	*a*
a	*c*

$A \bowtie_{A.Column\ B = C.Column\ B} (C)$
Output

Column A	Column B
a	a

Explanation: The data value "a" is available in both tables Hence we write that "a" is the table in the given output.

(c) Natural Join

Natural join is a type of inner join in which we do not need any comparison operators. In natural join, columns should have the same name and domain. There should be at least one common attribute between the two tables.
Example: Suppose there are two tables Table A and Table B
Table A

Number	Square
2	4
3	9

Table B

Number	Cube
2	8
3	27

$A \bowtie B$

Output

Number	Square	Cube
2	4	8
3	9	27

Explanation - Column Number is available in both tables Hence we write the "Number column once " after combining both tables.

2. Outer Join

Outer join is a type of join that retrieves matching as well as non-matching records from related tables. There are three types of outer join

- Left outer join
- Right outer join
- Full outer join

(a) Left Outer Join

It is also called left join. This type of outer join retrieves all records from the left table and retrieves matching records from the right table.

Example: Suppose there are two tables Table A and Table B

Table A

Number	Square
2	4
3	9
4	16

Table B

Number	Cube
2	8
3	27
5	125

$A \rtimes B$

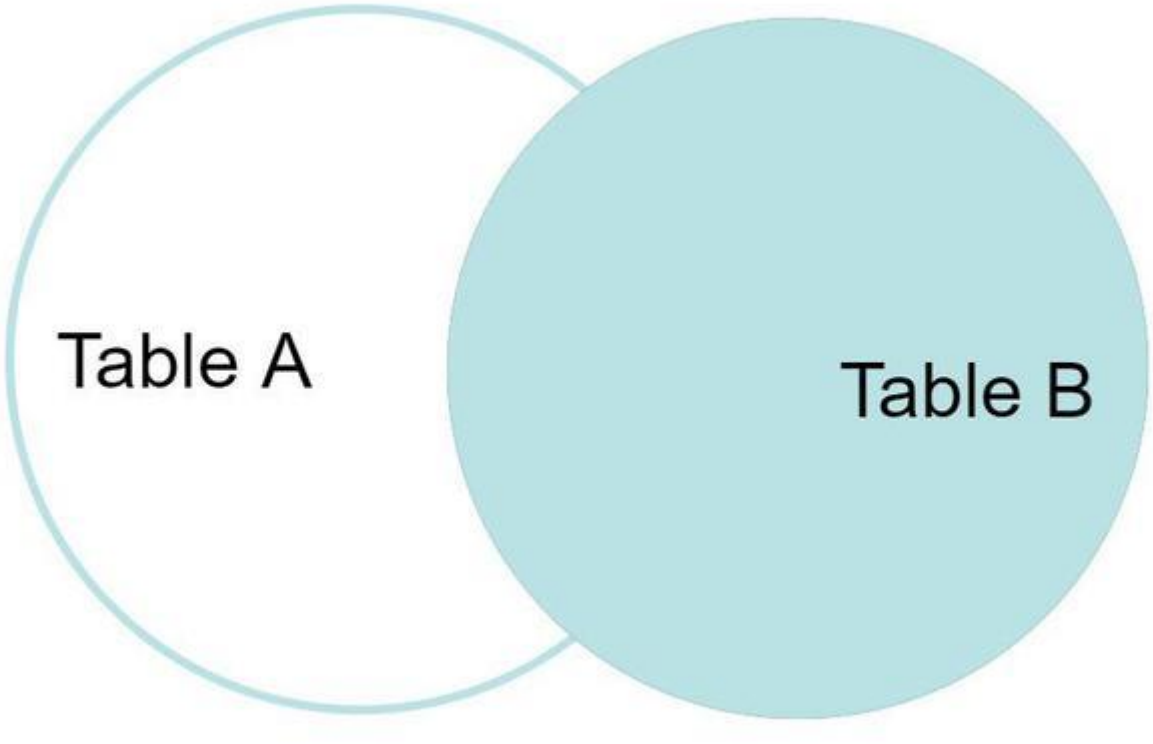
Output

Number	Square	Cube
2	4	8
3	9	27
4	16	NULL

Explanation: Since we know in the left outer join we take all the columns from the left table (Here Table A) In the table A we can see that there is no Cube value for number 4. so we mark this as NULL.

(b) Right Outer Join

It is also called a right join. This type of outer join retrieves all records from the right table and retrieves matching records from the left table. And for the record which doesn't lies in Left table will be marked as NULL in result Set.



Right Outer Join

Example: Suppose there are two tables Table A and Table B
$A \rtimes B$
Output:

Number	Square	Cube
2	4	8
3	9	27
5	NULL	125

Explanation: Since we know in the right outer join we take all the columns from the right table (Here Table B) In table A we can see that there is no square value for number 5. So we mark this as NULL.

(c) Full Outer Join

FULL JOIN creates the result set by combining the results of both LEFT JOIN and RIGHT JOIN. The result set will contain all the rows from both tables. For the rows for which there is no matching, the result set will contain *NULL* values.
Example: Table A and Table B are the same as in the left outer join

$A \not\bowtie B$

Output:

Number	Square	Cube
2	4	8
3	9	27
4	16	NULL
5	NULL	125

Explanation: Since we know in full outer join we take all the columns from both tables (Here Table A and Table B) In the table A and Table B we can see that there is no Cube value for number 4 and No Square value for 5 so we mark this as NULL.

SQL Triggers

A trigger in SQL is a special stored procedure that runs automatically when an INSERT, UPDATE, or DELETE operation occurs on a table. It helps automate actions and keep data consistent. Triggers are especially useful when we need to:

- Automatically update related tables
- Enforce business rules and data integrity
- Keep audit logs of data changes
- Perform actions when data is inserted, updated, or deleted

Example: Automatically Track When a User Record Is Updated

Step 1: Create Main Table

```
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100),
    updated_at TIMESTAMP
);
```

Step 2: Create Trigger

This trigger automatically updates the updated_at field whenever the user record is modified.

```
CREATE TRIGGER update_timestamp
BEFORE UPDATE ON users
FOR EACH ROW
```

```
BEGIN
   SET NEW.updated_at = CURRENT_TIMESTAMP;
END;
```

Step 3: Insert Sample Data

```
INSERT INTO users (id, name, email) VALUES (1, 'Amit', 'amit@example.com');
```

Output:

id	name	email	updated_at
1	Amit	amit@example.com	

Step 4: Update a Record

```
UPDATE users SET email = 'amit_new@example.com' WHERE id = 1;
```

Output:

id	name	email	updated_at
1	Amit	amit_new@example.com	2026-01-22 05:44:00

Syntax:

```
create trigger [trigger_name]
[before | after]
{insert | update | delete}
on [table_name]
FOR EACH ROW
BEGIN
END;
```

- The trigger has a name and runs on a specific table
- It activates before or after an insert, update, or delete action
- It runs SQL code automatically for each affected row

Types of SQL Triggers

Triggers can be categorized into different types based on the action they are associated with:

1. DDL Triggers

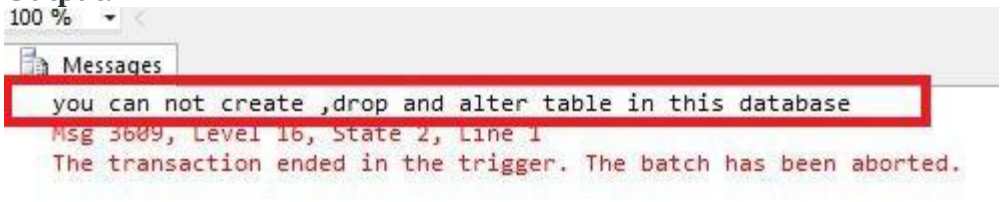
Data Definition Language (DDL) triggers run when commands like CREATE, ALTER, or DROP are used. They help track or stop changes to the database structure, such as creating, modifying, or deleting tables.

Example: Prevent Table Deletions

```
CREATE TRIGGER prevent_table_creation
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
```

```
BEGIN
  PRINT 'you can not create, drop and alter table in this database';
  ROLLBACK;
END;
```

Output:



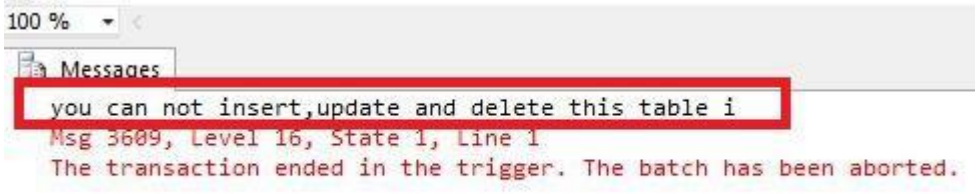
2. DML Triggers

DML triggers fire when we manipulate data with commands like INSERT, UPDATE, or DELETE. These triggers are perfect for scenarios where we need to validate data before it is inserted, log changes to a table, or cascade updates across related tables.

Example: To prevent unauthorized updates in a sensitive students table, we can create a trigger that blocks such changes.

```
CREATE TRIGGER prevent_update
ON students
FOR UPDATE, INSERT, DELETE
AS
BEGIN
   RAISERROR ('You can not insert, update and delete rows in this table.', 16, 1);
END;
```

Output:



DML Trigger

Note: *ROLLBACK TRANSACTION can included for safety, but in most cases RAISERROR itself will prevent the DML from completing.*

3. Logon Triggers

Logon triggers run when a user logs in. They are used to track logins, control access, and limit sessions. Messages from these triggers are stored in the SQL Server error log.

Example: Track User Logins

```
CREATE TRIGGER track_logon
ON LOGON
```

```
AS
BEGIN
  PRINT 'A new user has logged in.';
END;
```

Real-World Use Cases of SQL Triggers

1. Automatically Updating Related Tables (DML Trigger Example)

Triggers can automatically perform tasks when data changes in a table. For example, in a student database, when a student's grades are updated, the total score should also be updated automatically.

```
CREATE TRIGGER update_student_score
AFTER UPDATE ON student_grades
FOR EACH ROW
BEGIN
  UPDATE total_scores
  SET score = score + :new.grade
  WHERE student_id = :new.student_id;
END;
```

2. Data Validation (Before Insert Trigger Example)

Triggers can be used to validate data before insertion. For example, a trigger can check that grades are between 0 and 100 before allowing them to be stored in the table.

```
CREATE TRIGGER validate_grade
BEFORE INSERT ON student_grades
FOR EACH ROW
BEGIN
  IF :new.grade < 0 OR :new.grade > 100 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Invalid grade value.');
  END IF;
END;
```

The trigger checks if the inserted grade is valid. If not, it throws an error and prevents the insertion.

Viewing and Managing Triggers in SQL
We can use a query to list all triggers in SQL Server, which helps us manage and track triggers across multiple databases.

```
SELECT name, is_instead_of_trigger
FROM sys.triggers
WHERE type = 'TR';
```

- **name**: The name of the trigger.
- **is_instead_of_trigger**: Whether the trigger is an INSTEAD OF trigger.
- **type = 'TR'**: This filters the results to show only triggers.

BEFORE and AFTER Triggers
SQL triggers can be specified to run BEFORE or AFTER the triggering event.

- BEFORE Triggers run before an action (INSERT, UPDATE, DELETE) and are used for validation or changing values.
- AFTER Triggers run after the action and are used for logging or updating other tables.

Example: Using BEFORE Trigger for Calculations

In a Student Report Database, a trigger automatically calculates the total and percentage when a new record is added. A BEFORE INSERT trigger does this before saving the data.

Query:

```
mysql>>desc Student;
```

Output:

Field	Type	Null	Key	Default	Extra
tid	INTEGER	NO	PRI	NULL	auto_increment
name	TEXT	YES		NULL	
subj1	INTEGER	YES		NULL	
subj2	INTEGER	YES		NULL	
subj3	INTEGER	YES		NULL	
total	INTEGER	YES		NULL	
per	INTEGER	YES		NULL	

This SQL statement creates a trigger that calculates total and percentage before inserting student marks into the database. The trigger runs automatically whenever new data is added.

```
CREATE TRIGGER stud_marks
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
   UPDATE Student
   SET
      total = NEW.subj1 + NEW.subj2 + NEW.subj3,
      per = (NEW.subj1 + NEW.subj2 + NEW.subj3) * 60.0 / 100
   WHERE tid = NEW.tid;
END;
```

Output:

Trigger	Event	TableName	Timing	Created	Status
stud_marks	BEFORE	Student	BEFORE INSERT	2023-05-02 00:00:00	ACTIVE